

Hixean: cómo programar en él

Un compilador AOT a C11, sin máquina virtual

Hixean 0.3.0
octubre de 2026
github.com/wwwillcoxon/Hixean

Contenido

Antes de escribir una línea	3
El primer programa	3
Dónde van los archivos	3
Variables y tipos	5
Control de flujo	6
Funciones	7
Registros	8
Aritmética: verificada, y con intención explícita	9
Operadores propios	10
Bits	11
Cadenas	12
Arreglos	13
MAYBE, NIL, Result y ?	14
MATCH y patrones	15
Genéricos, restricciones y traits	16
Iteradores	17
Vectores	18
Módulos y paquetes	19

Antes de escribir una línea

Hixean compila a C11 y el C se compila con el compilador del sistema. No hay máquina virtual, ni recolector de basura, ni *unwinding*. Eso tiene una consecuencia práctica que conviene tener presente desde el principio: **lo que el programahace está en el C que ves cuando falla**. Un `hxc build` deja el generado en `build/gen/`, y se puede abrir.

El perfil por defecto es `freestanding`: `syscalls` directas, sin `libc`, memoria de la arena y del sistema. Un hola mundo ocupa 8 896 bytes. El perfil `libc` existe para cuando quieras `stdio` y compañía.

Un detalle que conviene saber antes que nada: **el perfil `freestanding` es de Linux x86-64**. Emite su propio `_start` y llama al kernel con `asm` en línea, así que el binario que produce solo arranca en Linux x86-64. El compilador en sí se compila en macOS y Windows, y allí el perfil por defecto es `libc`, que sí produce programas ejecutables. La puerta de 12 KiB se mide solo en Linux por eso mismo.

Los ejemplos de este documento se compilan y se ejecutan. El script `tools/verificar-guia.py` los saca del texto, los compila con `hxc` y compara su salida con la que dice aquí. Si un ejemplo dejara de compilar, la puerta fallaría.

El primer programa

Una extensión `.hxe` es un programa ejecutable; una `.hxf` es una biblioteca. El punto de entrada no se declara: es el archivo.

```
' hola.hxe
PRINT "Hola"
```

hola
~~salida~~

Para ejecutarlo:

```
hxc run hola.hxe
```

Para verlo en hexadecimal, que es la otra mitad de este lenguaje:

```
hxc build hola.hxe -o hola
./hola
```

Dónde van los archivos

| sufijo | qué es | |---|---| | `.hxe` | programa ejecutable | | `.hxf` | biblioteca de funciones exportables | | `.hxs` | módulo interno, para `IMPORT` | | `.hxt` | prueba: su salida esperada vive en `.hxt.out` | | `.hxx` | manifiesto de paquete | | `.hxq` | consulta de paquetes | | `.hxc` | biblioteca publicada, interfaz más código compilado |

Una prueba es un programa con su salida esperada al lado. Eso es todo lo que es una prueba en Hixean: no hay un `assert`, hay un fichero de texto.

```
' suma.hxt
PRINT 2 + 2
```

suma
~~salida~~

```
$ hxc test suma.hxt  
ok      suma.hxt
```

Variables y tipos

Se declaran con DIM, siempre antes de usarlas. Los tipos incorporados son BOOL, INT, I64, FLOAT, STRING y DURATION, más MAYBE T, ARRAY[T], REF T y PTR T.

```
DIM n AS INT = 42
DIM nombre AS STRING = "ana"
DIM ok AS BOOL = TRUE
DIM grande AS I64 = 90000000000
DIM x AS FLOAT = 2.5
```

Las variables son valores, no referencias: copiar copia. Un TYPE es un registro y también se copia. El verificador es estricto en una cosa que sorprende viniendo de otros lenguajes: **detrás de un `MAYBE` hay que mirar**

```
DIM Perhaps AS MAYBE STRING = "hola"
DIM vacio AS MAYBE STRING = NIL

IF Perhaps.IsNil THEN
  PRINT "no hay nada"
ELSE
  PRINT "hay: " ++ Perhaps.Or("nada")
END IF
```

hay: hola

Un T se convierte solo a MAYBE T. Al revés no: un MAYBE STRING donde se espera un STRING es un error, y con razón, porque el valor puede no estar.

Control de flujo

IF con ELSEIF y ELSE. Después de THEN puede venir una línea o un bloque.

```
DIM n AS INT = 7

IF n < 0 THEN
    PRINT "negativo"
ELSEIF n == 0 THEN
    PRINT "cero"
ELSE
    PRINT "positivo"
END IF
```

positivo

salida

WHILE y FOR. El FOR tiene TO y STEP, y el cuerpo va hasta NEXT.

```
DIM i AS INT = 0
WHILE i < 3
    PRINT i
    i = i + 1
WEND

FOR j = 0 TO 9 STEP 3
    PRINT j
NEXT j
```

0

salida

1

2

0

3

6

9

BREAK sale del bucle, CONTINUE pasa a la siguiente vuelta, EXIT sale de la función.

Funciones

```
FUNCTION Doble(x AS INT) AS INT
  RETURN x * 2
END FUNCTION

PRINT Doble(21)
```

~~salida~~
42

Un parámetro puede tener valor por defecto, y entonces se puede omitir:

```
FUNCTION Saluda(nombre AS STRING, saludo AS STRING = "Hola") AS STRING
  RETURN saludo ++ ", " ++ nombre
END FUNCTION

PRINT Saluda("ana")
PRINT Saluda("ana", "Buenos días")
```

~~salida~~
Hola, ana
Buenos días, ana

REF es un préstamo explícito: quien recibe promete devolverlo. Se escribe `DIM d AS REF C = k` al declarar, y reasignar la variable después es un error (E0408), porque el préstamo ya está en otro sitio.

Registros

Un TYPE es un registro con nombre: sus campos llevan tipo, y se inicializan con DIM. No tiene métodos; los métodos son de los traits, que es lo que se ve más abajo.

```
TYPE Punto
  x AS FLOAT
  y AS FLOAT
END TYPE

DIM a AS Punto
a.x = 3.0
a.y = 4.0

PRINT a.x.ToString() ++ ", " ++ a.y.ToString()
```

~~3.0, 4.0~~
3.0, 4.0

Un TYPE **no** admite métodos dentro. Si vienes de un lenguaje donde un registro lleva sus propias funciones, saber dónde está la frontera ayuda: en Hixean el dato es el registro y el comportamiento es un TRAIT que un tipo implementa. Es una separación deliberada, y evita media tabla virtual.

Para el comportamiento hay dos caminos. Uno es el operador propio del que hemos hablado. El otro, cuando lo que quieres es añadir una misma operación a varios tipos:

```
TYPE Punto
  x AS FLOAT
  y AS FLOAT
END TYPE

TRAIT ConNombre
  METODO Nombre(quien AS SELF) AS STRING
END TRAIT

IMPLEMENTAR Punto PARA ConNombre
  METODO Nombre(quien AS SELF) AS STRING
    RETURN "punto"
  END METHOD
END IMPLEMENTAR

FUNCTION Presentacion(p AS Punto) AS STRING
  RETURN "soy un " ++ ConNombre.Nombre(p)
END FUNCTION

DIM a AS Punto
PRINT Presentacion(a)
```

~~soy un punto~~
soy un punto

El despacho es estático: IMPLEMENTAR se resuelve al compilar, no hay tabla virtual, y el cuerpo de la función llama al método por el trait.

Aritmética: verificada, y con intención explícita

La suma y la resta sobre enteros **comprueban el desbordamiento** y abortan con el código 70. Es una decisión, no un descuido: en un lenguaje de sistemas es mejor un programa que para que saber que el número no cabía.

| operadores | qué hacen | |---|---| | + - | verificados: trampa y salida 70 si desbordan | | * / MOD | sin verificar, como en C | | +% -% | envolvente: dan la vuelta | | + | - | * | | saturante: se quedan en el máximo o el mínimo |

```
DIM a AS INT = 2000000000
PRINT a +% a
```

salida 204967296

Con + ese mismo programa no imprimiría nada: $2000000000 + 2000000000$ no cabe en un INT de 32 bits, así que aborta con el código 70 y un mensaje en la salida de error. Probalo, es la forma más rápida de entender la decisión.

Si lo que quieres es el resultado del entorno que al dar la vuelta, pídeselo con el operador que lo dice: `a +% a`. Escribir + y esperar que no compruebe, o escribir +% esperando que pare, son los dos errores que esta separación evita.

La división por cero se comprueba también, y con divisor constante el error es de compilación: es mejor no generar código que aborta.

Operadores propios

Un TYPE puede definir cómo se comporta con los operadores del lenguaje. El mangling es `op_<nombre>__<Tipo>`, así que dos sobrecargas del mismo signo con tipos distintos conviven.

```
TYPE Fraccion
  num AS INT
  den AS INT
END TYPE

FUNCTION OPERATOR + (a AS Fraccion, b AS Fraccion) AS Fraccion
  DIM r AS Fraccion
  r.num = a.num * b.den + b.num * a.den
  r.den = a.den * b.den
  RETURN r
END FUNCTION

FUNCTION OPERATOR == (a AS Fraccion, b AS Fraccion) AS BOOL
  RETURN a.num * b.den == b.num * a.den
END FUNCTION

FUNCTION OPERATOR < (a AS Fraccion, b AS Fraccion) AS BOOL
  RETURN a.num * b.den < b.num * a.den
END FUNCTION

DIM media AS Fraccion
media.num = 1
media.den = 2
DIM tercio AS Fraccion
tercio.num = 1
tercio.den = 3

DIM suma AS Fraccion = media + tercio
PRINT suma.num.ToString() ++ "/" ++ suma.den.ToString()
PRINT media == tercio
PRINT media < tercio
```

~~5/6~~
5/6
false
false

Se admiten + - * / MOD ++ == <> < <= > >= y los de aritmética explícita (+% -% +| -| *|).
!= es otra forma de <>: sobrecargar los dos es un error, porque son el mismo operador.

El tipo del primer parámetro manda. `a * 6` con `OPERATOR * (a AS Fraccion, b AS INT)` funciona. Un parámetro que no sea de un TYPE declarado da E0215.

Bits

AND, OR y XOR leen de dos maneras según el tipo de los operandos: con BOOL son los operadores booleanos, y con INT o I64 son la operación de bits. ~ es el complemento a bits, y << y >> mueven posiciones.

```
DIM mascara AS INT = 255
DIM permisos AS INT = 5

IF (permisos AND 4) == 4 THEN PRINT "puede ejecutar"
IF (permisos AND 1) == 1 THEN PRINT "puede leer"

DIM h AS INT = 12345
h = h XOR 255
h = ~h
PRINT h
```

```
puede ejecutar
puede leer
-12487
```

Mezclar un BOOL con un entero es un error (E0307), con una nota que dice que a un lado le falta el otro. El lenguaje no convierte un BOOL a entero por su cuenta, ni al revés.

Y ojo con la precedencia, que es la de C: == se ata más fuerte que AND, así que `x AND 1 == 1` se lee como `x AND (1 == 1)`. Los paréntesis no son decorativos ahí.

El desplazamiento está comprobado: con cuenta constante lo dice el verificador (E0316) y con cuenta variable lo dice el runtime, que aborta con el mismo código 70 que el desbordamiento de la suma. El C no promete nada en ninguno de los dos casos.

Cadenas

STRING es una estructura con puntero y longitud: los literales se comparan por contenido y no se pueden modificar en su sitio. ++ concatena.

```
DIM s AS STRING = "Hola"
PRINT s ++ ", " ++ "mundo"
PRINT s.Len()
PRINT s.Upper()
PRINT s.At(1)
```

```

HOLA, mundo
4
HOLA
0
```

Los métodos de STRING son Len, IsEmpty, Upper, Lower, Trim, Slice, At y Repeat.
Para poner un número dentro de un texto está ToString(), que existe en INT, I64, FLOAT, BOOL y DURATION:

```
DIM n AS INT = 42
PRINT "n = " ++ n.ToString()
```

```

n = 42
```

El texto que devuelve vive en memoria propia y **no se libera** Hixean no tiene recolector y no va a fingir uno. Quien llame muchas veces en un bucle debe encerrarlo en un ARENA.
La biblioteca estándar de texto es un módulo más:

```
IMPORT std.texto

PRINT texto.StartsWith("hola mundo", "hola")
PRINT texto.EndsWith("hola mundo", "mundo")
PRINT texto.PadLeft("7", 3, "0")
```

```

true
true
007
```

Arreglos

Un arreglo tiene su tamaño en el tipo, y por eso `Len()` es una constante: no cuesta código.

```
DIM a AS INT[5]
FOR i = 0 TO 4
    a[i] = i * i
NEXT i

PRINT a.Len()
PRINT a[3]
PRINT a.At(3)
```

Salida
5
9
9

`a[i]` es rápido y **no compruebanada**, como en casi todos los lenguajes. `a.At(i)` comprueba el índice y aborta con un número si no cabe. Quien quiera el diagnóstico usa `At`.

El arreglo dinámico se declara `ARRAY[T]` o `T[]`, y crece con `Push`:

```
DIM d AS ARRAY[INT]
DIM i AS INT = 0
WHILE i < 5
    d.Push(i * 10)
    i = i + 1
WEND

PRINT d.Len()
PRINT d.At(4)
d.Set(0, 99)
PRINT d[0]
```

Salida
5
40
99

Cuando crece no copia el arreglo entero con `realloc`: reserva un bloque nuevo y copia. `realloc` no existe en el perfil freestanding, y copiar a mano es predecible. La memoria sale de una arena, así que un `Push` dentro de un bucle que se repite mucho crece la arena; para eso está `ARENA`.

MAYBE, NIL, Result y?

MAYBE T es un valor de tipo T o nada. NIL solo vale dentro de un MAYBE: fuera de él es un error (E0211), no un cero disfrazado. El acierto se ve con `IsNil()`, se sustituye con `Or(x)` y se transforma con `Map(f)`.

`Result<T,E>` es lo que casi todo lenguaje llama `Either` o `Result`, con dos parámetros: el del éxito y el del error, que aquí es `STRING`.

```
FUNCTION Divide(a AS INT, b AS INT) AS Result<INT, STRING>
  IF b == 0 THEN RETURN Err("division por cero")
  RETURN Ok(a)
END FUNCTION
```

El `?` toma el valor del `Ok` o propaga el error: es el camino corto para no escribir el mismo `CASE Err` en cada función. Fíjate en `Result<INT, STRING>`: un `Result` sin los dos parámetros no sabe qué lleva dentro, y tanto `?` como `CASE Ok(v)` lo dicen con E0219 en vez de adivinar.

Un `MATCH` sobre un `Result` cubre `Ok` y `Err` y no necesita `CASE ELSE`:

```
FUNCTION Divide(a AS INT, b AS INT) AS Result<INT, STRING>
  IF b == 0 THEN RETURN Err("division por cero")
  RETURN Ok(a)
END FUNCTION
```

```
FUNCTION Describe(n AS INT) AS STRING
  MATCH Divide(n, 0)
  CASE Ok(v) THEN
    RETURN "ok"
  CASE Err(m) THEN
    RETURN "error de " ++ m.ToString() ++ " caracteres"
  END MATCH
END FUNCTION
```

```
FUNCTION Raiz(n AS INT) AS Result<INT, STRING>
  DIM mitad AS INT = Divide(n, 2)?
  RETURN Ok(mitad * mitad)
END FUNCTION
```

```
PRINT Describe(4)
```

`error de division por cero caracteres`

En cualquier otro tipo, el `MATCH` necesita `CASE ELSE` (E0405).

MATCH y patrones

```
FUNCTION Clasifica(c AS INT) AS STRING
  MATCH c
    CASE 1 | 2 | 3 THEN
      RETURN "bajo"
    CASE n WHEN n > 100 THEN
      RETURN "muy alto"
    CASE n WHEN n > 10 THEN
      RETURN "alto"
    CASE ELSE THEN
      RETURN "normal"
  END MATCH
END FUNCTION

PRINT Clasifica(2)
PRINT Clasifica(50)
```

bajo
normal
alto

El guard ve lo que el patrón liga, así que CASE n WHEN n > 10 usa la n que ese mismo CASE acaba de ligar. Un patrón puede traer un rango (CASE 1 TO 5), y | varias alternativas. Un CASE con dos alternativas que ligan el mismo nombre es E0217: el cuerpo no podría saber de cuál de las dos habla. Un CASE por alternativa se entiende mejor.

Genéricos, restricciones y traits

Un TYPE o una FUNCTION genérica se declara con <T>, y DONDE exige un trait. La comprobación ocurre al instanciar, en el punto de la llamada, no dentro del cuerpo.

```
TRAIT Sumable
  METODO Valor(quien AS SELF) AS INT
END TRAIT

FUNCTION Suma<T>(v AS ARRAY[T]) AS INT DONDE T: Sumable
  DIM total AS INT = 0
  DIM i AS INT = 0
  WHILE i < v.Len()
    total = total + v.At(i).Valor
    i = i + 1
  WEND
  RETURN total
END FUNCTION
```

No hay herencia y hay subtipado estructural: dos TYPE con la misma forma son compatibles donde se pida el más pequeño, sin declarar una interfaz por cada interfaz.

Los traits se despachan de forma estática: no hay tabla virtual ni coste en tiempo de ejecución, porque la resolución es al compilar.

Iteradores

FOR ... IN con un iterador perezoso. Los verbos son pocos a propósito: Rango, RangoF, Map, Filter, Take y First.

```
FOR i IN Rango(1, 4)
  PRINT i * i
NEXT i

FOR c IN Rango(1, 10).Filter(FUNC(n AS INT) AS BOOL
  RETURN n MOD 2 == 0
END).Take(3)
  PRINT c
NEXT c
```

salida

4
9
2
4
6

Una FUNC se eleva a una función del módulo y se referencia como puntero a función. **Captura el entorno por valor.** una lambda puede leer una variable de su alrededor, y lo que lee es una copia del valor que tenía cuando se construyó la lambda, no una referencia. Por eso dos MAP sobre el mismo dato con la misma lambda no se pisan entre sí, y por eso una lambda que no captura no cuesta nada: sigue siendo una función normal.

Se acepta donde el lenguaje ya sabe que espera una función, que es MAP, FILTER y FOLD. Guardar una lambda en una variable y llamarla más tarde sigue sin poder hacerse, porque no hay un tipo que describa una función; es lo que queda abierto en ADR 0008.

Vectores

VEC2, VEC3 y VEC4 son FLOAT agrupados: tres FLOAT en la pila, sin puntero ni indirección. Se escriben con un literal y se imprimen como sus componentes.

```
DIM a AS VEC3 = (1.0, 2.0, 3.0)
DIM b AS VEC3 = (4.0, 5.0, 6.0)
```

```
PRINT a
PRINT a.DOT(b)
PRINT a.CROSS(b)
PRINT a.LEN()
PRINT a.ADD(b)
PRINT a.NORMALIZED()
PRINT a.x
PRINT a.zy
```

Cada verbo va en dos formas, la de método y la de función: `a.DOT(b)` es `DOT(a, b)`. Un nombre de dos a cuatro letras es un *swizzle*: `a.zy` es un VEC2 con los componentes en el orden escrito. MAT4 y QUAT existen en el runtime pero **no tienen nombre** no hay manera de construirlos, y dar el nombre sin constructor sería una promesa que el lenguaje no puede cumplir.

Módulos y paquetes

Un módulo importable es un `.hxs`. El namespace es el último segmento del nombre: `IMPORT std.texto` se usa como `texto`.

```
MODULE utiles

EXPORT FUNCTION Doble(x AS INT) AS INT
  RETURN x * 2
END FUNCTION

END MODULE
```